Abstract: Datenbank für eine Buchtausch-App

1 Zielsetzung und Konzept

Das Ziel der Arbeit ist ein relationales Datenbankschema für eine Buchtausch-App, über die Nachbarn private Bücher anbieten und ausleihen. Dieses Schema soll den Bestand nach bibliografischen Merkmalen und nach Entfernung durchsuchbar machen, die zeitliche Verfügbarkeit einzelner Exemplare abbilden, den Ausleihvorgang bis zur Rückgabe festhalten und durch gegenseitige Bewertungen Vertrauen zwischen unbekannten Beteiligten schaffen. Die geltenden Regeln sollen idealerweise Bedingungen der Datenbank sein und nicht der Anwendung überlassen bleiben.

Das Konzept trennt den bibliografischen Titel (*Buchtitel*) vom physischen *Exemplar*, sodass ein Buch bei mehreren Besitzern erfasst wird, ohne bibliografische Angaben zu wiederholen. Der *Standort* eines Nutzers trägt Geokoordinaten für die Umkreissuche, Abholfenster sind als *Zeitslot* gehalten, und eine *Ausleihe* verknüpft Exemplar, Zeitslot, Ausleiher und Verleiher. Das Modell umfasst zwölf Entitäten und eine Zuordnungstabelle und weicht an sechs dokumentierten Stellen bewusst von der dritten Normalform ab.

2 Methodik und Umsetzung

Das Vorgehen entsprach den drei Phasen des Portfolios. In der Konzeptionsphase entstanden aus einer Anforderungsspezifikation das Entity-Relationship-Modell in Martin-Notation, gezeichnet mit draw.io, und das Datenwörterbuch. Das Feedback hob die Normalisierung und die Integritätsregeln hervor und empfahl für Phase 2 den Schwerpunkt auf positive und negative Testfälle. Die Erarbeitungsphase setzte das Modell als Schema in PostgreSQL 18 um, füllte es mit Dummy-Daten und testete es mit psql. In der Finalisierungsphase wurden die Normalisierung geprüft, drei Regeln ergänzt oder korrigiert, die Metadaten erhoben und das Ergebnis in einem GitHub-Repository bereitgestellt. Die Berichte entstanden mit LaTeX.

PostgreSQL wurde gewählt, weil zwei Regeln über *CHECK* und *UNIQUE* hinausgehen. Weder die Abholfenster noch die Verleihzeiträume eines Exemplars dürfen sich überschneiden, was *EXCLUDE*-Constraints über Bereichstypen mit der Erweiterung *btree_gist* voraussetzt (PostgreSQL Global Development Group, 2026a, 2026c). Zusammengesetzte Fremdschlüssel erzwingen, dass der Abholort dem Besitzer und der Zeitslot dem Exemplar gehört, und zwei Trigger sichern Regeln über mehrere Tabellen. Die Umkreissuche berechnet die Entfernung nach dem sphärischen Kosinussatz mit einer Vorauswahl über den Breitengrad (Veness, 2020, 2022). Insgesamt 95 Testfälle, davon 51 negative und 44 positive, weisen die Regeln nach. Eine Prozedur führt jeden Test aus, protokolliert den SQLSTATE und nimmt die Änderung zurück.

3 Ergebnis und Zielerreichung

Das Schema umfasst 13 Tabellen mit 163 Einträgen, 19 Fremdschlüssel, 14 *CHECK*-Bedingungen, 9 Eindeutigkeitsbedingungen, 2 Ausschluss-Constraints, 2 Trigger und 5 zusätzliche Indizes. Sechs Abfragen decken unter anderem die Umkreissuche und die Joins über die drei Dreifachbeziehungen ab. Seit der Finalisierungsphase erzwingt die Datenbank auch, dass eine Bewertung erst nach der Rückgabe erfolgt und das Ausleihdatum im gewählten Zeitslot liegt. Das in der Konzeption formulierte Ziel ist damit mit drei Einschränkungen erreicht. Das Modell weicht an sechs begründeten Stellen von der dritten Normalform ab, die Trigger erkennen spätere Änderungen an einer bereits geprüften

Ausleihe nicht, und die Umkreissuche berechnet die Luftlinie, ohne die Zeitslots zu berücksichtigen.

4 Reflexion

Die größte Herausforderung war der Entwurf des ER-Modells ohne Vorlage. Trotz Vorkenntnissen aus der Tätigkeit als Entwickler waren Korrekturen nötig. Die Trennung von Buchtitel und Exemplar ergab sich erst bei der Prüfung auf Redundanzfreiheit, zwei Integritätslücken bei Abholort und Zeitslot fielen im zweiten Durchgang auf, und die Finalisierungsphase deckte zwei bisher nicht ausgewiesene transitive Abhängigkeiten auf. Eine Aussage zur Normalform ist daher erst belastbar, wenn jede funktionale Abhängigkeit einzeln geprüft ist. In einem neuen Projekt würde der Verfasser diese Abhängigkeiten deshalb bereits in der Konzeption je Tabelle auflisten.

Regeln über mehrere Tabellen sind als Trigger umgesetzt, da ein *CHECK*-Constraint nur die geprüfte Zeile auswertet und ein Fremdschlüssel einen Besitzerwechsel blockiert hätte. Dafür prüft ein Trigger nur beim Schreiben seiner eigenen Tabelle (PostgreSQL Global Development Group, 2026a).

Ein zweiter Schluss betrifft die Tests. Die 88 Testfälle der Erarbeitungsphase prüften jede Regel einzeln, aber keinen Ablauf der App. Ein Test der Folge Ausleihen, vorzeitig Zurückgeben und erneut Ausleihen hätte früher gezeigt, dass ein vorzeitig zurückgegebenes Exemplar bis zum Ende der maximalen Ausleihdauer gesperrt blieb. Der Fehler wurde erst in der Finalisierungsphase behoben. Ebenso hätten Dummy-Daten mit Standorten innerhalb einer Stadt die Umkreissuche aussagekräftiger geprüft als drei Großstädte.

Literaturverzeichnis

PostgreSQL Global Development Group. (14. Mai 2026a). *5.5. Constraints*. PostgreSQL 18 Documentation. `https://www.postgresql.org/docs/18/ddl-constraints.html`

PostgreSQL Global Development Group. (14. Mai 2026c). *F.8. btree_gist — GiST operator classes with B-tree behavior*. PostgreSQL 18 Documentation. `https://www.postgresql.org/docs/18/btree-gist.html`

Veness, C. (2020). *Selecting points from a database by latitude/longitude within a bounding circle*. Movable Type Scripts. `https://www.movable-type.co.uk/scripts/latlong-db.html`

Veness, C. (2022). *Calculate distance, bearing and more between latitude/longitude points*. Movable Type Scripts. `https://www.movable-type.co.uk/scripts/latlong.html`